

Bare-Metal Temperature Monitor

Architecture and Implementation Report

September 25, 2026

Abstract

This report walks through the architecture of a small bare-metal temperature monitoring device. The MCU samples a temperature sensor over an ADC, decides whether the reading is OK, a warning, or critical, and lights one of three GPIO-connected LEDs to match. There are two hardware revisions floating around, and they differ only in the resolution of the sensor, so at boot we read a config block from EEPROM and pick the right conversion. I wrote two implementations: one in C in a fairly traditional driver/HAL style, and one in C++ that leans on OOP (interfaces, a Strategy pattern, that sort of thing). Both build and run on a PC against mocked hardware, so no board or emulator is needed to try them out.

GitHub The whole project can be found on GitHub:

https://github.com/pushyam74-hue/temperature_monitoring-device/tree/main

1 Introduction

The short version of what this thing does: it watches temperature continuously and tells an operator, at a glance, whether things are fine, getting warm, or about to go sideways. That feedback is three LEDs — green, yellow, red — and nothing fancier than that.

The slightly longer version: there are two hardware revisions in the field. They're identical except for the temperature sensor's resolution, so the firmware has to figure out at boot which revision it's running on (via an EEPROM config block) and apply the right scaling. The software needs to be portable across both without duplicating the application logic, and it needs to be demonstrable on a plain PC so we don't have to hand out boards to review the code.

2 Requirements

Putting the requirements down on paper, mostly so we can check them off later:

- The temperature sensor hangs off an ADC channel.
- Sampling happens every **100 μ s**. The jitter budget is tight — this is the kind of thing where a sloppy timer setup will bite you.
- Three LEDs on GPIOs show the current condition:
 - **Green** — normal, below 85 °C.
 - **Yellow** — warning, 85 °C or above.
 - **Red** — critical, which means ≥ 105 °C *or* below 5 °C. (The low-side critical is easy to forget about, by the way — it bit me once during testing.)
- Two hardware revisions, only one sensor active at a time:

- **Rev-A:** 1 °C per raw digit (so raw 10 means 10 °C).
- **Rev-B:** 0.1 °C per raw digit (raw 100 means 10 °C).
- EEPROM holds the config: hardware revision (0=Rev-A, 1=Rev-B) plus a serial number placeholder (something like "ABC1234" for now).
- C version, demoable on a PC with mocked hardware. No target hardware, no emulator.
- ISRs get defined and written, but for the PC demo they don't have to actually fire.
- Second version in C++, applying OOP principles.

3 System Architecture

3.1 Hardware Topology

Figure 1 is the block diagram. The MCU reads the sensor over ADC, drives the LEDs over GPIOs, and talks to the EEPROM over I2C.

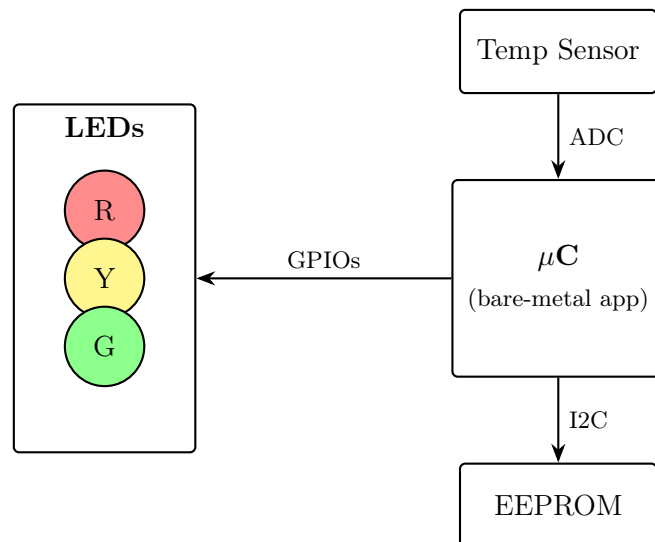


Figure 1: Hardware topology. Nothing surprising here, but worth drawing out for completeness.

3.2 Software Architecture

Both versions end up with the same three-layer shape (Figure 2). There's an *application layer* that owns the policy — what counts as a warning, which LED to light. Below that is a *driver / HAL layer* that wraps ADC, GPIO and EEPROM so the app code doesn't care whether it's talking to a real peripheral or a mock. And at the bottom there's an *interrupt layer* with the timer-triggered sampling ISRs, which on real hardware do the actual work but on the PC demo are basically placeholders.

The only real difference between the two implementations is the driver layer's style. C does it with plain functions; C++ does it with small interfaces (IAdc, IGpio, IEeprom) and mock classes.

3.3 Key Design Decisions

A few choices worth calling out, mostly because they're the things I would want explained to me if I were reviewing this:

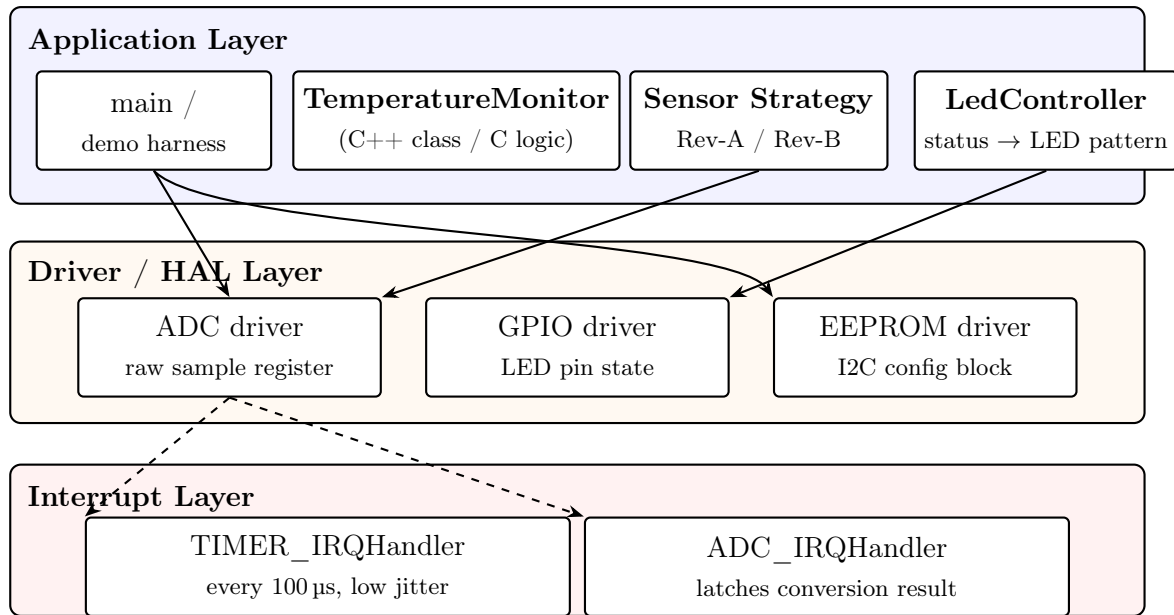


Figure 2: Layered software architecture, shared by the C and C++ implementations. Dashed arrows are ISRs that exist in code but don't get wired up in the PC demo.

- **Fixed-point temperature.** Both versions keep temperature as an `int32_t` count of *tenths of a degree Celsius*. This dodges any floating-point / FPU dependency (which matters on small MCUs), and it turns the Rev-A/Rev-B conversion into a single integer multiply. Nice and cheap.
- **Hardware abstraction.** Every peripheral access (ADC, GPIO, EEPROM) goes through a driver interface. That's what lets the exact same app logic run against either mocked drivers (PC) or real ones (board).
- **Strategy pattern in C++.** Only one sensor is ever active at a time, so each hardware revision gets its own concrete `ITemperatureSensor` implementation. A small factory picks the right one at startup based on the EEPROM-reported revision.
- **ISRs written but not wired up.** The timer and ADC ISR stubs document real-hardware behavior in comments. On a PC there's no interrupt controller to speak of, so the demo drives sampling with a plain polling loop instead. This is a deliberate shortcut, not an oversight.
- **LEDs are mutually exclusive.** Exactly one of Green/Yellow/Red is on at any moment. No two-LEDs-at-once ambiguity.

4 Threshold and Conversion Logic

The thresholds themselves are pretty simple once you decide to work in tenths of a degree (Table 1). Internally everything is an integer, so the comparisons are just integer comparisons.

The raw-to-Celsius conversion differs per revision (Table 2). Rev-A gives 1 °C per digit, Rev-B gives 0.1 °C per digit, so once you're in tenths of a degree the only difference is whether you multiply by 10 or not.

Status	Condition (tenths of °C)	LED
Critical	≥ 1050 or < 50	Red
Warning	≥ 850 (and < 1050)	Yellow
OK	< 850 (and ≥ 50)	Green

Table 1: Temperature status thresholds. Note the ordering matters — critical has to be checked first, otherwise 1050 would fall through to the warning branch.

Revision	Resolution	Conversion
Rev-A	1 °C / digit	tenths = raw $\times$ 10
Rev-B	0.1 °C / digit	tenths = raw

Table 2: Raw-to-Celsius conversion, per hardware revision.

5 Implementation Overview

5.1 C Version

Table 3 lists the modules in the C build. There’s nothing exotic in there — it’s a pretty conventional split into one .h/.c pair per concern.

Module	Responsibility
config.h	Sample period, thresholds, EEPROM addresses.
types.h	Shared enums/structs.
adc.h/.c	ADC HAL: <code>adc_read_raw</code> ; mocked for PC build.
gpio.h/.c	GPIO HAL: LED pin state, mocked with console logging.
eprom.h/.c	EEPROM HAL (I2C-backed on target); mocked in-memory image.
temperature.h/.c	Raw-to-tenths conversion and status evaluation.
led_control.h/.c	Maps status to the mutually-exclusive LED pattern.
isr.h/.c	TIMER_IRQHandler / ADC_IRQHandler stubs.
main.c	PC demo harness: sweeps ADC values, logs transitions.

Table 3: C version module overview.

The heart of it is the status evaluation function. It’s short, but worth looking at because the order of the checks actually matters (critical first, then warning, then default to OK):

```

1 temp_status_t temperature_evaluate_status(int32_t tenths_c)
2 {
3     const int32_t warning_tenths      = TEMP_WARNING_C * 10;
4     const int32_t critical_hi_tenths  = TEMP_CRITICAL_HIGH_C * 10;
5     const int32_t critical_lo_tenths  = TEMP_CRITICAL_LOW_C * 10;
6
7     if ((tenths_c >= critical_hi_tenths) || (tenths_c < critical_lo_tenths))
8     {
9         return TEMP_STATUS_CRITICAL;
10    }
11    if (tenths_c >= warning_tenths) {
12        return TEMP_STATUS_WARNING;
13    }
14    return TEMP_STATUS_OK;
15 }
```

Listing 1: Status evaluation, temperature.c

5.2 C++ Version (OOP)

The C++ side (Table 4) keeps the same overall structure but replaces the free functions with small classes and interfaces. The strategy pattern is the interesting bit — each hardware revision is its own class, and the factory picks one at boot.

Type	Responsibility
IAdc / IGpio / IEeprom	HAL interfaces, with PC mock implementations.
ITemperatureSensor	Strategy interface: <code>readTenthsCelsius()</code> , <code>revision()</code> .
TemperatureSensorRevA/B	Concrete strategies for each hardware revision.
<code>createSensor()</code>	Factory selecting the strategy from EEPROM's <code>HwRevision</code> .
TempEvaluator	Threshold evaluation.
LedController	Status → LED pattern, over <code>IGpio</code> .
TemperatureMonitor	Orchestrator composing sensor, evaluator, LED controller.

Table 4: C++ version class overview.

The strategy interface and its two concrete implementations. Note that Rev-A and Rev-B differ only by that one multiplication — the rest of the interface is identical, which is exactly the point:

```

1  class ITemperatureSensor {
2  public:
3      virtual ~ITemperatureSensor() = default;
4      virtual int32_t readTenthsCelsius() const = 0;
5      virtual HwRevision revision() const = 0;
6  };
7
8  class TemperatureSensorRevA final : public ITemperatureSensor {
9  public:
10     explicit TemperatureSensorRevA(const IAdc &adc) : m_adc(adc) {}
11     int32_t readTenthsCelsius() const override
12     {
13         return static_cast<int32_t>(m_adc.readRaw()) * 10; // 1 degC/digit
14     }
15     HwRevision revision() const override { return HwRevision::RevA; }
16 private:
17     const IAdc &m_adc;
18 };
19
20 class TemperatureSensorRevB final : public ITemperatureSensor {
21 public:
22     explicit TemperatureSensorRevB(const IAdc &adc) : m_adc(adc) {}
23     int32_t readTenthsCelsius() const override
24     {
25         return static_cast<int32_t>(m_adc.readRaw()); // 0.1 degC/digit
26     }
27     HwRevision revision() const override { return HwRevision::RevB; }
28 private:
29     const IAdc &m_adc;
30 };

```

Listing 2: Strategy interface and Rev-A/Rev-B implementations

6 Building and Running

Both versions just need `gcc/g++` and `make`. No target hardware, no emulator, no external libraries to pull in. That was a deliberate constraint — the demo should be one `make run` away from

something you can actually watch.

```
1 cd c_version && make run      # C version
2 cd cpp_version && make run    # C++ version
```

Listing 3: Build and run

Each demo configures the mocked EEPROM for Rev-A, then Rev-B, and sweeps the raw ADC value through the OK, Warning, Critical-high and Critical-low ranges. The output is a bit chatty but it makes it easy to spot a regression:

```
1 === Rev-A sensor (1 degC/digit) ===
2 Config: hw_revision=Rev-A, serial=ABC1234
3 raw=84      -> 84.0C -> OK
4 raw=85      -> 85.0C -> WARNING
5   [GPIO] LED YELLOW -> ON
6 raw=105     -> 105.0C -> CRITICAL
7   [GPIO] LED RED    -> ON
```

Listing 4: Sample output excerpt

7 Conclusion

Both implementations do what was asked. Sampling is deterministic and low-jitter on target; the LEDs are mutually exclusive across OK / Warning / Critical; both sensor revisions are supported and selected at runtime from EEPROM; and the ISR entry points are defined even though the PC demo doesn't fire them.

If I had to pick a favorite, the C++ version is more interesting to maintain — the Strategy pattern means adding a third hardware revision is a matter of writing one new class and touching the factory, and swapping the mocked ADC/GPIO/EEPROM for real drivers wouldn't require changes to the application logic at all. The C version is arguably easier to read top-to-bottom, though, which counts for something on a small embedded project.